

Functions that have input arguments and/or local variables can be called only from one of the before mentioned program threads.

4.2 Types

C is strictly typed language, which means that every object, function, and expression needs to have a strictly defined type, known in the time of compilation. Note that C works exclusively with numeric types.

The type serves:

- to determine the correct memory allocation required initially.

- to interpret the bit patterns found in the object during subsequent access.

- in many type-checking situations, to ensure that illegal assignments are trapped.

CCS supports many standard (predefined) and user-defined data types, including signed and unsigned integers in various sizes, floating-point numbers in various precisions, arrays, structures, and unions. In addition, pointers to most of these objects can be established and manipulated in memory.

The type determines how much memory is allocated to an object and how the program will interpret the bit patterns found in the object's storage allocation. A given data type can be viewed as a set of values (often implementation-dependent) that identifiers of that type can assume, together with a set of operations allowed on those values. The compile-time operator, `sizeof`, lets you determine the size in bytes of any standard or user-defined type.

The CCS standard libraries and your own program and header files must provide unambiguous identifiers (or expressions derived from them) and types so that CCS can consistently access, interpret, and (possibly) change the bit patterns in memory corresponding to each active object in your program.

4.2.1 Type categories

Common way to categorize types is to divide them into:

- fundamental

- derived

The fundamental types represent types that cannot be separated into smaller parts. They are sometimes referred to as unstructured types. The fundamental types are `void`, `char`, `int`, `float`, and `double`, together with `short`, `long`, `signed`, and `unsigned` variants of some of these.